

A still from the movie 'The Hobbit' showing a character in a goat suit running towards a character in a white dress. The character in the goat suit is on the left, and the character in the white dress is on the right. The background is a misty, outdoor setting.

James' Super Cool IS 201 Cheat Sheet

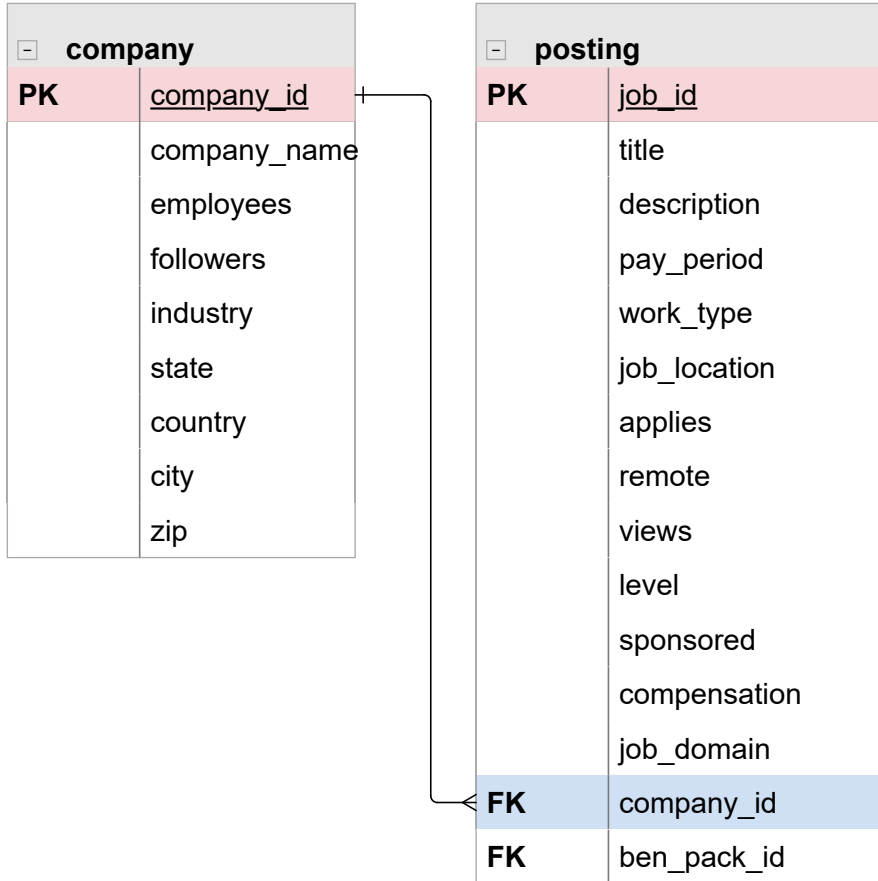
You



Index

Slide	Topic
2	<u>ERDs</u>
6	<u>SQL</u>
9	<u>Flow Charts</u>
10	<u>VBA</u>
16	<u>Statistics</u>
17	<u>Tableau</u>
18	<u>Solver</u>
19	<u>HTML</u>
28	<u>CSS</u>

ERD: Primary / Foreign Keys

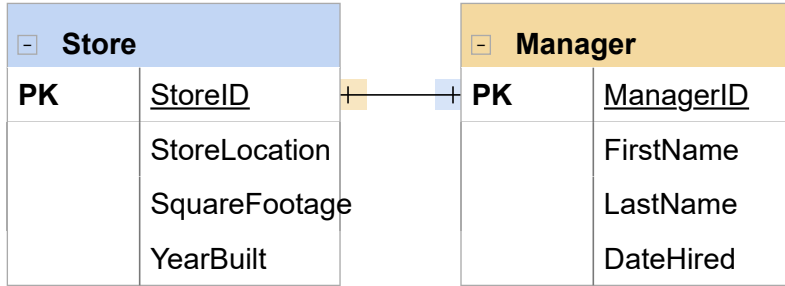


A **primary key** is a unique identifier (think social security number)

A **foreign key** is another table's primary key, used to link two tables together

Entity
Relationship
Diagram

ERD: Cardinality

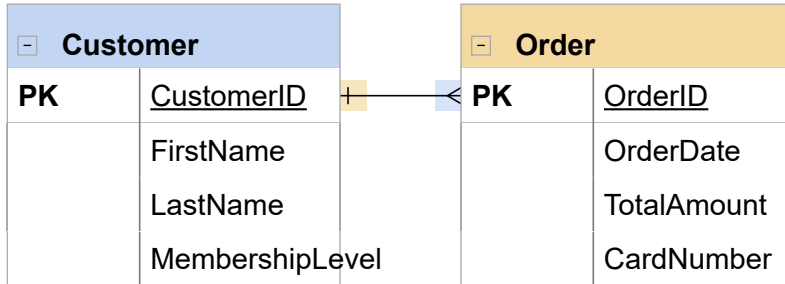


Read left-to-right AND right-to-left

One to One (1:1)

A store has one manager

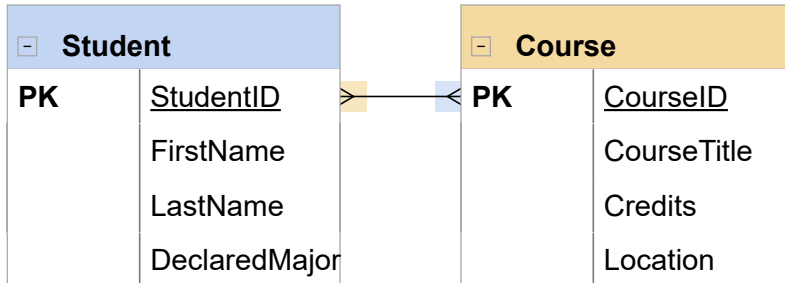
A manager works at one store



One to Many (1:N)

A customer can have multiple orders

An order belongs to one customer



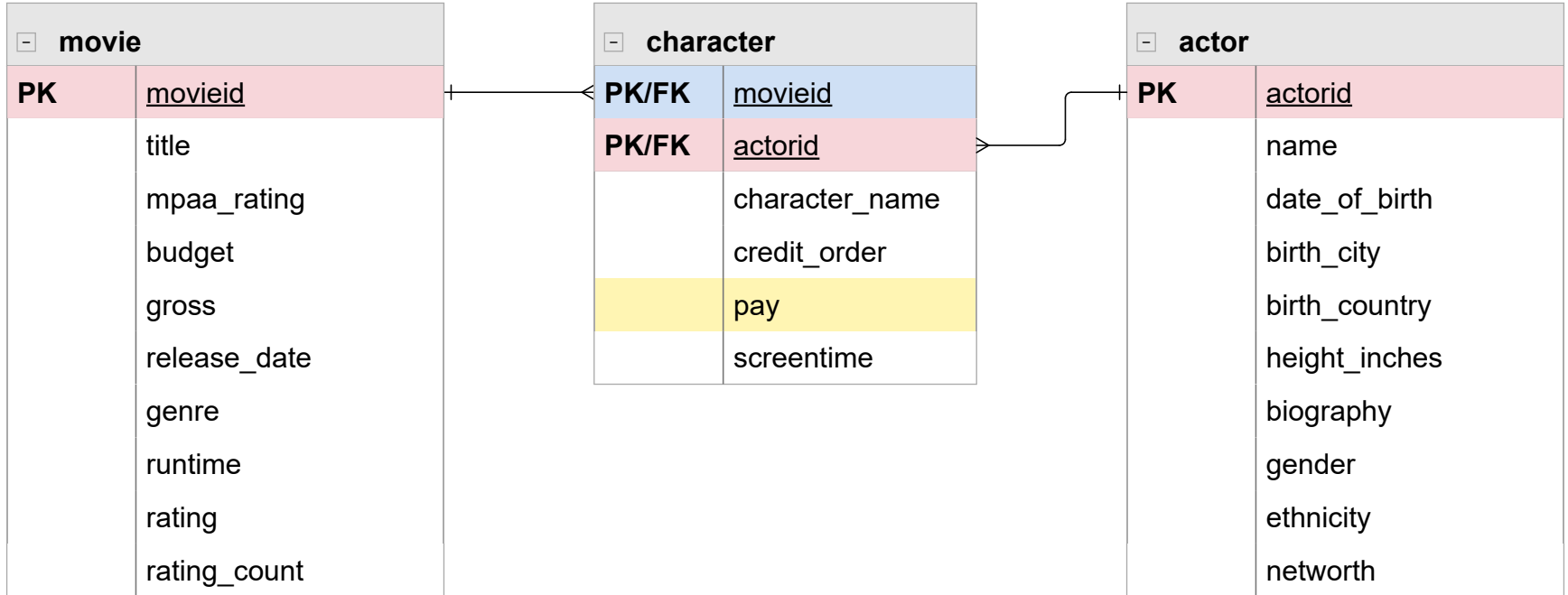
Many to Many (M:N)

A student can enroll in many courses

A course can have lots of students

Note! M:N cardinality requires a composite table (next slide)

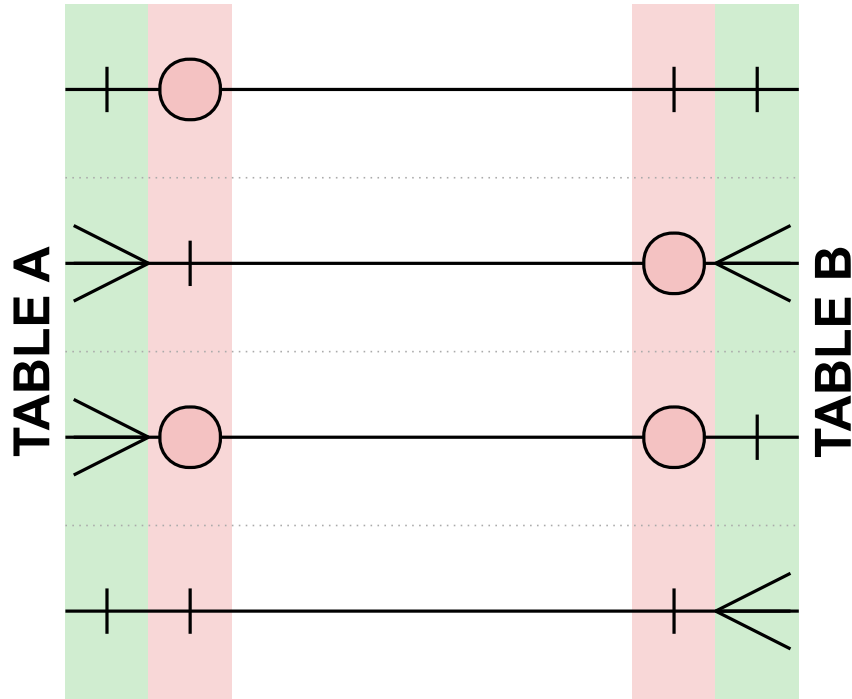
ERD: Composite Table



To know how much money Tom Hanks got **paid** to play Woody in *Toy Story*, you need to know both the **movie** and the **actor**. His pay is probably different than when he was Woody in *Toy Story 2*, or when he was Forrest Gump in *Forrest Gump*

ERD: Minimum / Maximum Cardinality

The inner marks are minimum (0 or 1)
and the outer marks are maximum (1 or many)



A could have 1 B or 1 B

B could have 0 A or 1 A

A could have 0 B or many B

B could have 1 A or many A

A could have 0 B or 1 B

B could have 0 A or many A

A could have 1 B or many B

B could have 1 A or 1 A

Think: a student could have 0 cars, but they could also have multiple.

go to gaskination.com/sql

SQL: Clauses

```
1 SELECT      -- attributes -- SELECT * for all, SELECT DISTINCT for unique results
2 FROM        -- tableA
3 JOIN        -- tableB ON tableA.attribute = tableB.attribute (order doesn't matter)
4 WHERE       -- attribute filters [=, !=, <>, IS, LIKE '%__%', IN ("__","__","__")]
5     AND/OR  -- you only say WHERE once, but can have many filters
6 GROUP BY    -- all non-aggregated attributes when aggregating
7 HAVING      -- group-based filters (not used in the SQL project)
8 ORDER BY    -- attributes in ASC (default) / DESC order
9 LIMIT       -- to xx top results
10
11 -- aggregate(attribute) AS newName
12 count()
13 avg()
14 min()
15 max()
16 -- etc.
```

Structured
Query
Language

SQL: Where (1/2)

Text

in quotes

ta_name = "James"
→ exactly "James"

ta_name != "James"
ta_name <> "James"
→ anything but "James"

ta_name LIKE "%ame%"
→ contains "ame"

% = any number of characters
_ = exactly one character

ta_name IN ("James", "Robert", "Frankie")
→ exact match for any of these

Blank

no quotes

ta_name IS NULL
→ blank / missing (any type)

ta_name IS NOT NULL
→ has data

SQL: Where (2/2)

Numbers

no quotes

hours > 10 hours < 10

→ greater than / less than

hours BETWEEN 5 AND 10

→ 5 to 10 (includes both)

hours >= 10 hours <= 10

→ ... or equal to

Booleans

no quotes

is_ta = TRUE

→ only the TAs

is_ta = FALSE

→ everyone else

Dates

in quotes:

"YYYY-MM-DD"

hire_date > "2024-01-01"

→ after Jan 1, 2024

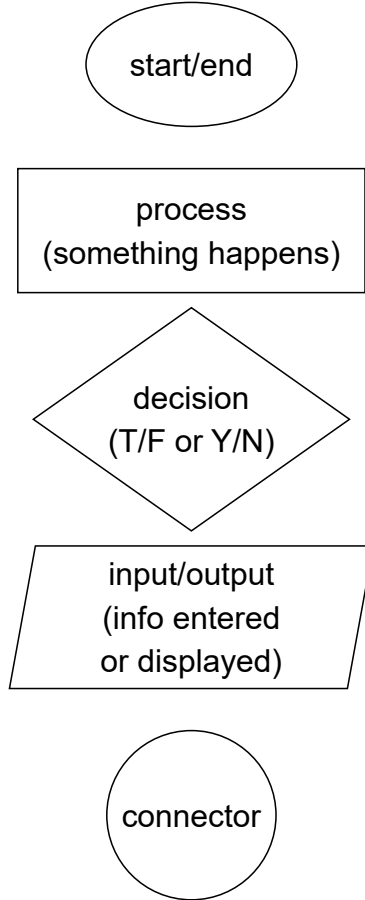
hire_date < "2024-01-01"

→ before Jan 1, 2024

hire_date BETWEEN "2024-01-01" AND "2024-12-31"

→ sometime in 2024

Flow Charts

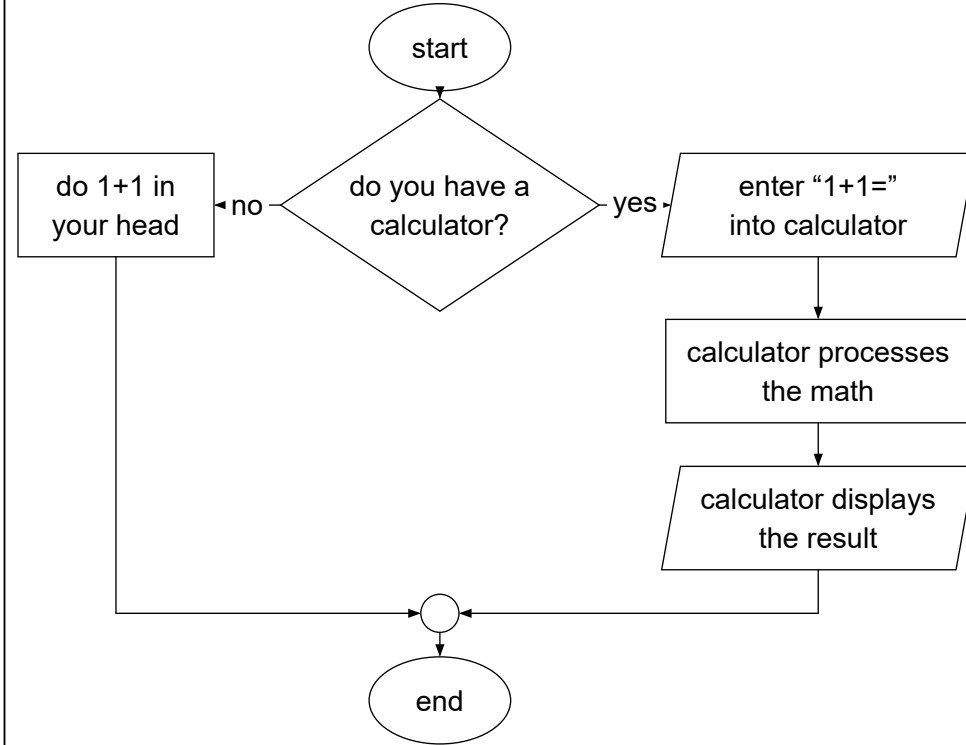


you can only have ONE start
but you CAN have multiple
ends

decisions are the ONLY
thing that can have more
than one arrow pointing OUT
of them

connectors are the ONLY
thing that can have more
than one arrow pointing
INTO them

EXAMPLE: How to solve 1+1



VBA: Basics

Option Explicit 'This makes it so that you can only use variables you've declared (VERY RECOMMENDED)

```
Sub thisIsMySubName()
```

```
    'BASICS!
```

```
    'Generally you will first refer to an object type (like a Sheet)
```

```
    'and then tell it what you want to do (like delete or add or copy, etc):
```

```
        ' Sheets and Worksheets are the same thing
```

```
        Sheets("MySheet").Delete
```

```
        Sheets.Add.name = "MySheet"
```

```
        Worksheets("MySheet").Activate
```

```
        ' Range and Cells are SIMILAR but different: Range("E7") = Cells(7, 5)
```

```
        Range("A1").Activate
```

```
        ActiveCell.Value = "Hello"
```

```
        Range("A1:D3").Copy
```

```
        Range("E5").PasteSpecial
```

```
        ' Columns and Rows are what they sound like
```

```
        Columns("B:D").Delete
```

Visual
Basic for
Applications

VBA: Navigation and Misc.

'NAVIGATION!

```
'  
ActiveCell.End (xlDown)  
ActiveCell.End (xlToLeft)  
  
' Move (rows, columns) based on a cell or range  
ActiveCell.Offset(1, 2).Value = "NewSpot"  
    Range("A1").Offset(3, 4).Value = "NewSpot" 'The cell that now says "NewSpot" is E4
```

' MISCELLANEOUS!

```
' Bolding  
Range("A1").Font.Bold = True  
  
' Autofitting columns  
Columns.AutoFit 'all columns  
Columns("A:C").AutoFit 'specific columns  
  
' Currency format  
Range("A1").Style = "Currency"  
  
' Bottom border  
Range("A1").Borders(xlEdgeBottom).LineStyle = xlContinuous  
  
' Text splicing  
name = "James"  
first_initial = Left(name, 1)
```

VBA: Variables

```
'DECLARE VARIABLES!
Dim i As Integer    'a whole number
Dim j As Double     'a number with a decimal
Dim k As String     'a line of text
Dim l As Boolean    'true or false
Dim m As Worksheet

i = 5
j = 1.2345
k = "Hello there! General Kenobi!"
l = True
Set m = Sheets("Sheet1") 'objects (sheets, books, ranges) use Set instead of =

'USER INTERFACE!

'Tell the user something
MsgBox "Oh hello there user, welcome to my spreadsheet"

'Ask for an input
Dim username As String
username = InputBox("What's your name?")

' COMBINING STRINGS

Dim myOutput As String
Dim name As String
name = "James"

' COMbine strings with ampersands (&)-- don't forget spaces
myOutput = "Hello. My name is " & name & " and this is Disney Channel!"
```

'CONDITIONALS!

```
' If Statements
If i < 0 Then
    ' your code here
ElseIf i < 5 Then
    ' your code here
ElseIf i < 10 Then
    ' your code here
Else
    ' your code here
End If
```

```
'Case Statements
Select Case fruit
    Case "apple", "strawberry"
        Color = "red"
    Case "grape"
        Color = "purple"
    Case Else
        Color = "unknown"
End Select
```

```
'Another Case Statement
Select Case temperature
    Case Is >= 100
        feel = "too hot"
    Case 80 To 100
        feel = "pretty warm"
    Case Is < 80
        feel = "I'm probably staying out of the pool"
End Select
```

VBA: Conditionals

'LOOPS!

VBA: Loops

```
'For Loop
For i = 0 To 5
    ' your code here
Next i
```

```
'For Each Loop
Dim x As Range
For Each x In Range("A2", Cells(lastrow, "A"))
    ' your code here
Next
```

```
'For Each Loop 2
Dim x As Range
Dim mtRange As Range
Set myRange = Range("A1:E5")
For Each x In myRange
    ' your code here
Next
```

```
'Do While Loop
Do While i = 5 'condition does NOT have to be numbers
    ' your code here
Loop
```

```
' Do Until Loop
Do Until i = 10 'condition does NOT have to be numbers
    'your code here
Loop
```

End Sub

VBA: Functions

```
Function thisIsMyFunctionName(num As Integer, tf As Boolean) As String
```

```
' a function declaration is just like a set of dims  
' the INPUT is in the parentheses ()  
' the OUTPUT is outside of it  
' so this function KIND OF has this baked into it:
```

```
Dim num As Integer
```

```
Dim tf As Boolean
```

```
Dim thisIsMyFunctionName As String
```

```
' the INPUT then comes from your excel sheet  
' for example, in your sheet you cell might say:  
' =thisIsMyFunctionName(15, True)
```

```
' the OUTPUT comes from your function  
' set the function's name equal to something  
' in this case, our output has to be a string:
```

```
thisIsMyFunctionName = "That's a very nice hat you have there, partner!"
```

```
' generally your inputs will be used to calculate/decide your output
```

```
End Function
```

Statistics

TEST & USE CASE

Correlation

numeric – numeric
(Pearson r coefficient)

Regression

numeric – multi. numeric

ANOVA

numeric – mult. categorical

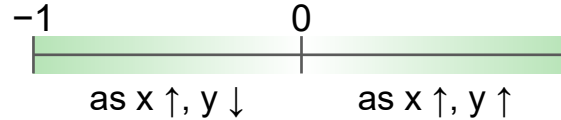
T-test

numeric – 2 pair categorical

OUTPUT

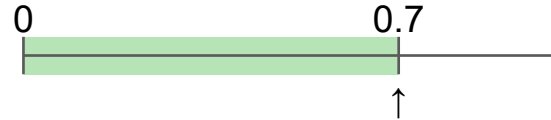
HOW TO READ IT

→ r



strength = distance from 0; sign = direction

→ R^2



70% of the variance in y is explained by the x 's

usually 0.05

→ p



reject the null fail to reject the null
significant **not significant**

*p works like a percentage:
0.05 = 5% to see this or more extreme*

Null Hypothesis

H_0 : No x 's are significant

Alt Hypothesis

H_A : At least one x is significant

Scientific Notation

$aE^b = a \times 10^b$

$aE3 = a \times 1000$ (big)

$aE-3 = a \times 0.001$
(small)

Tableau

“Default” means leave it alone

If the assignment says “default,” you don’t have to change anything. It’s the setting Tableau starts with.

Toolbar buttons worth knowing



Swap rows and columns: flips the x- and y-axis. A sideways chart can tell a clearer story.



Sort ascending: smallest to largest



Sort descending: largest to smallest

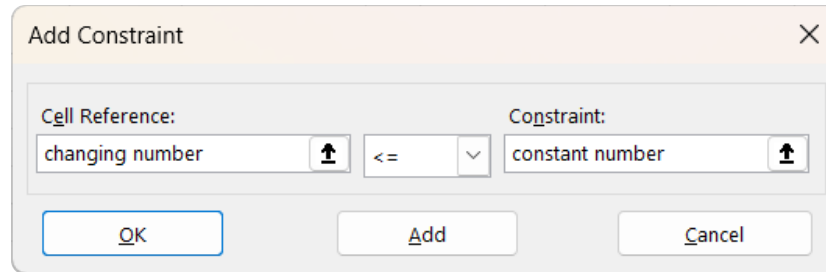
Solver

Two important Excel formulas:

=sum(A1:A5) → $A1 + A2 + A3 + A4 + A5$

=sumproduct(A1:A5, B11:B15) → $A1*B11 + A2*B12 + \dots + A5*B15$
the two ranges should be the same size and shape

Constraints:



Add Constraint

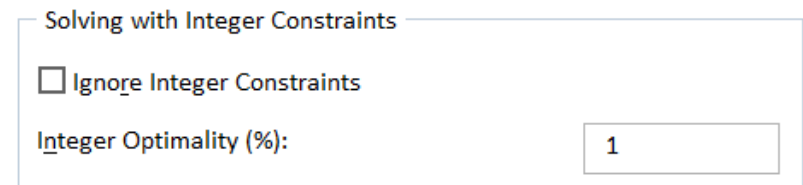
Cell Reference: changing number 

Constraint: $\leq$ constant number 

OK Add Cancel

If integer isn't working:

Solver > Options > uncheck this box:



Solving with Integer Constraints

☐ Ignore Integer Constraints

Integer Optimality (%): 1

HTML: Setup

In an html file inside VS Code,
type an exclamation point and enter to automatically generate:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>

</body>
</html>
```

Go here for free bootstrap themes:
startbootstrap.com/themes/portfolio-resume

HyperText
Markup
Language

HTML: Tags

```
<!-- This is a comment -->
<h1>Main heading</h1>
<h2>Section heading</h2>
<h3>Subsection</h3>

<p>This is a paragraph.</p>

<!-- Line Break -->
<br>
```

```
<!-- Unordered List (Bullets) -->
<ul>
    <li>Item 1</li>
    <li>Item 2</li>
    <li>Item 3</li>
</ul>

<!-- Change <ul> to <ol>
for an ordered list (Numbers) -->
```

Every tag follows this pattern:

<tag>	← start tag
stuff	← what the tag applies to
</tag>	← end tag: same name, plus a /

The start and end tags act like parentheses.

(a few, like
, have no end tag)

HTML: Anchors

```
<!-- Hyperlinks -->
```

```
<a href="https://www.google.com">Visit Google</a>
```

```
<!-- Folder References -->
```

```
<a href="otherpage.html">Visit Other Page</a>
```

```
<a href="subfolder/page.html">Visit Subfolder Page</a>
```

```
<a href="../anotherpage.html">Visit Another Page</a> ← ../ goes up one folder
```

```
<!-- On-page Anchors -->
```

```
<h1 id="top">This is the main heading</h1>
```

```
<a href="#top">Jump to the top</a> ← # + the id jumps to it on the same page
```

HTML: Images

```
<!-- Images -->  
  
  

```

Notice: images do NOT have an end tag `</img>`
(neither do line breaks `</br>`)

HTML: Divisions

Divs don't inherently do anything: they're invisible boxes you put content into so that you can isolate it for positioning and CSS styling using classes.

HTML:

```
<div class="square-image">
  
</div>
```

CSS:

```
.square-image {
  width: 200px;
  height: 200px;
}
```

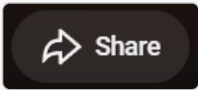
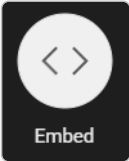
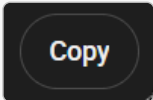
For more, see:

[CSS: Selectors/Properties/Values](#)

HTML: Embedding

Embeds sometimes don't show up until your site is live

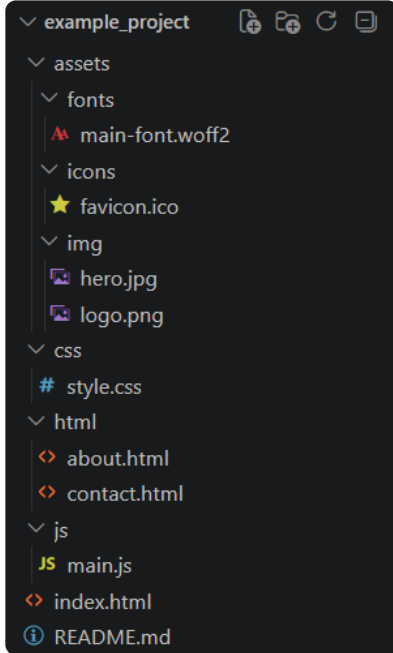
YouTube

- 1 Under the video, click **Share** 
- 2 Click **Embed** 
- 3 Click **Copy** 
- 4 Paste it into your HTML

Tableau

- 1 Go to public.tableau.com/app/discover
- 2 Search for a relevant viz
- 3 Click the share icon 
- 4 Click **Copy Embed Code** 
- 5 Paste it into your HTML

HTML: Best Practices



Use folders!

Give each type of file its own folder: `css/`, `html/`, `js/`, and `assets/` (with `img/`, `fonts/`, `icons/` inside).
`index.html` stays at the top level. It's your homepage, so it's the first page the site loads.

Never use spaces in file or folder names

Use a dash or underscore instead:

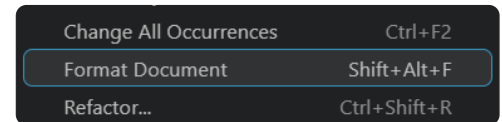
✓ `about-me.html` ✗ `about me.html`
✓ `example_project/` ✗ `example project/`

Format your code

Fixes indentation and long lines.

Right click → **Format Document**

shortcut: `Shift+Alt+F`



HTML: Uploading to GitHub

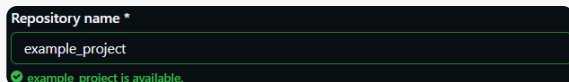
A. Create the repository

1 Go to github.com and sign in (or make a free account)

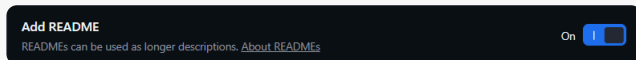
2 Click **New**



3 Type a **Repository name** (no spaces!)

A dark input field with the text "example_project" and a green checkmark below it with the text "example_project is available."

4 Leave it **Public** and turn **Add README** on

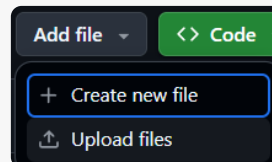
A dark button with the text "Add README" and a toggle switch on the right that is turned on. Below the button, it says "READMEs can be used as longer descriptions. About READMEs".

5 Click **Create repository** at the bottom



B. Upload your website

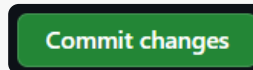
6 On your new repository's page, click **Add file** → **Upload files**



Open your project folder and drag everything *inside* it (index.html, css/, ...)

7 onto the page
Upload the CONTENTS of your project, not the folder itself

8 Click **Commit changes** ("commit" = save)

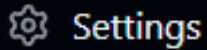


Next: turn it into a live website → [Going Live](#)

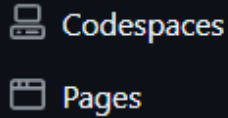
HTML: Going Live

On your repository's page (from the last slide):

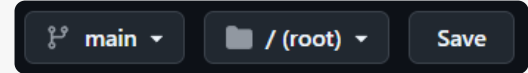
- 1 Click the **Settings** tab at the top



- 2 In the left sidebar, click **Pages**



- 3 Under **Branch**, change None to main, then click **Save**



- 4 Wait a couple of minutes, then **refresh** the page. Your website's link will appear at the top:

Your site is live at https://jimna-h.github.io/example_project/
Last **deployed** by  [jimna-h](#) 1 minute ago

 Visit site

Unpublish site

CSS: Basics

1. Connect it to your HTML

```
<head>
  <link rel="stylesheet" href="css/style.css">
</head>
```

Goes inside the `<head>` of every page.
`href` is the path to your `.css` file.

2. Every rule looks like this

```
selector {
  property: value;
  property2: value2;
}
```

property = what to change, **value** = what to change it to. That's all CSS is! (see next slide)

3. It's cascading: the later rule wins

```
p {
  color: blue;
}
p {
  color: red;
}
```

If you paint a door blue and then re-paint it red, it will be red instead of blue. CSS works the same way, but with everything not just colors.

Cascading
Style
Sheets

CSS: Selectors / Properties / Values

Built-in selectors

Use a tag name (body, h1, p, a, img, ...) to style every one of those tags.

```
body { ← every <body>
  background-color: #f0f0f0;
  font-family: Arial, sans-serif;
  margin: 0;
  padding: 20px;
}
```

Class (custom) selectors

Make your own: a **period** + any name.
Styles only tags with that class.

```
.highlight {
  background-color: yellow;
  font-weight: bold;
}
```

Apply it in your HTML (no period):

```
<p class="highlight">Hi!</p>
```

There are A LOT of selectors and properties. Google and AI are your friend for finding exactly what you want!

jimna-h.github.io/james_super_cool_is_201_cheatsheet



Suggestions? Reach out to me at jbruce1@byu.edu

IS 201 TA Lab